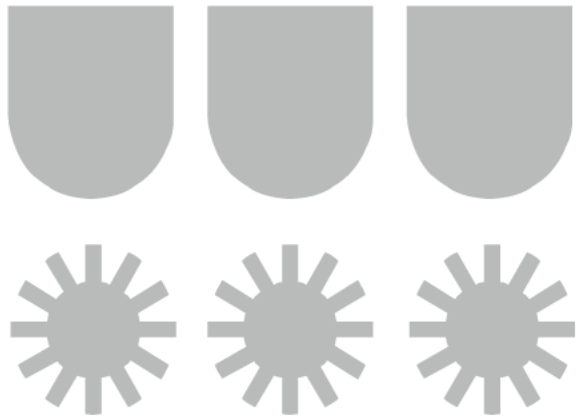




UNIVERSIDAD DEL BÍO-BÍO

Introducción a la Programación

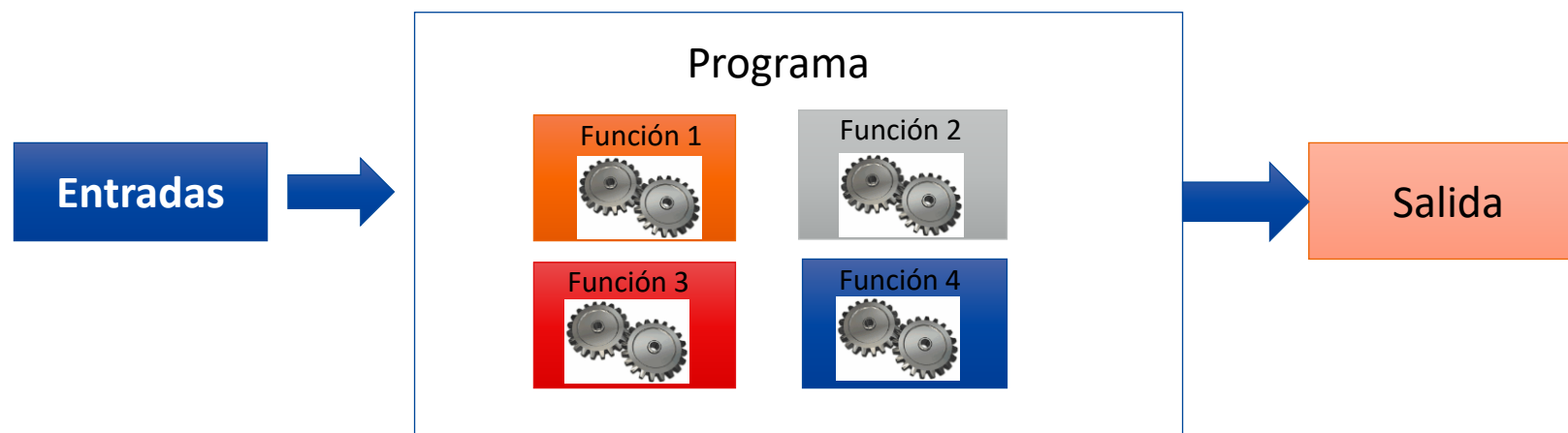
Introducción a Funciones en C



¿Qué es una función?



- Una función es un bloque de código que realiza una operación específica
- Permite mantener la modularidad en el código del programa
- Facilita la comprensión del código fuente
- Ayuda a no repetir líneas de código de operaciones recurrentes





¿Qué es una función?

```
1  #include<stdio.h>
2  #include<math.h>
3  int main()
4  {
5      int x;
6      scanf("%d",&x);
7
8      printf("Valor Ingresado %d \n", x);
9
10     float raiz;
11     raiz = sqrt(x);
12     printf("Raiz cuadra es %f",raiz);
13 }
```

Función que permite ingreso de datos

Función que permite mostrar por pantalla

Función que retorna la raíz cuadrada de un número





Estructura de una función

- ❖ **Tipo_de_dato:** Indica el tipo de dato que la función retorna al finalizar su ejecución
- ❖ **nombre_función:** nombre definido para identificar la función. Como buena práctica el nombre debe ser descriptivo y breve
- ❖ **Argumentos:** Son los datos que necesita la función para ejecutar el código de esta.
- ❖ **Cuerpo de la función:** son las instrucciones o código que la función realiza

```
Tipo_de_dato nombre_funcion( argumentos )  
{  
    Cuerpo de la función  
    return (variable o valor);  
}
```

- ❖ **return:** corresponde al valor que devuelve la función. La función termina cuando se encuentra la instrucción *return*. En funciones de tipo void esta instrucción es opcional





Estructura de una función

```
Tipo_de_dato nombre_funcion( argumentos )  
{  
    Cuerpo de la función  
    return (variable o valor);  
}
```

```
int sumatoria(int n)  
{  
    int i, sum=0;  
    for(i=0; i<=n; i++)  
        sum+=i;  
    return (sum);  
}
```

```
int sumatoria()  
{  
    int i, sum=0, n;  
  
    printf("Ingrese n: ");  
    scanf("%d", &n);  
    for(i=0; i<=n; i++)  
        sum+=i;  
    return (sum);  
}
```

Las funciones pueden o no recibir argumentos,
eso dependerá del problema que deben
resolver

Ejemplo: definición de una función

```
1 #include<stdio.h>
2
3 int sumatoria(int n);
4
5 int main()
6 {
7     int n,r;
8     scanf("%d",&n);
9     r=sumatoria(n);
10    printf("resultado es: %d",r);
11 }
```

Prototipo de función

Llamado o invocación de función

Función

```
12
13 int sumatoria(int n)
14 {
15     int i,sum=0;
16     for(i=0;i<=n;i++)
17         sum+=i;
18     return (sum);
19 }
```



Ejemplo: llamado o invocación

```
1 #include<stdio.h>
2
3 int sumatoria(int n);
4
5 int main()
6 {
7     printf("resultado es: %d \n", sumatoria(4));
8     printf("resultado es: %d \n", sumatoria(7));
9     printf("resultado es: %d \n", sumatoria(10));
10 }
```

Llamado a función **sumatoria**
dentro de **printf()**

```
12
13 int sumatoria(int n)
14 {
15     int i, sum=0;
16     for(i=0; i<=n; i++)
17         sum+=i;
18     return (sum);
19 }
```





Ejemplo: Llamado o invocación (modificar)

```
1 #include<stdio.h>
2
3 float sumatoria(int n);
4
5 int main()
6 {
7     printf("suma = %f \n", sumatoria(4));
8     printf("suma = %f \n", sumatoria(7));
9     printf("suma = %f \n", sumatoria(10));
10 }
11
```

Función modificada para
entregar otro resultado
ahora de tipo **float**

```
12 float sumatoria(int n)
13 {
14     float i, sum=0;
15     for(i=1; i<=n; i++)
16         sum+= (i+(2*n))/(2*i);
17     return (sum);
18 }
19
```



Ejemplo Funciones

Validar edad de una persona

```
int ingresar_edad()  
{  
    int edad;  
    do  
    {  
        scanf("%d",&edad);  
    }  
    while(edad<0 || edad > 120);  
    return edad;  
}
```



```
int ingresar_edad()  
{  
    int edad;  
    do  
    {  
        printf("Ingresa edad: ");  
        scanf("%d",&edad);  
        if(edad<0 || edad > 120)  
            printf("ERROR: edad fuera de rango [0,120]\n");  
    }while(edad<0 || edad > 120);  
    return edad;  
}
```





Ejemplo Funciones

Validar edad de una persona

```
1  #include<stdio.h>
2
3  int ingresar_edad();
4
5  int main()
6  {
7      int edad_1, edad_2;
8
9      edad_1=ingresar_edad();
10     edad_2=ingresar_edad();
11
12     printf("La edad 1 es %d y la edad 2 es %d",edad_1,edad_2);
13 }
```

```
15 int ingresar_edad()
16 {
17     int edad;
18     do
19     {
20         printf("Ingresa edad: ");
21         scanf("%d",&edad);
22
23         if(edad<0 || edad > 120)
24             printf("ERROR: edad fuera de rango [0,120]\n");
25
26     }while(edad<0 || edad > 120);
27
28     return edad;
29 }
```

Código disponible en Moodle.
Archivo "ejemplo_validar_edad.c"





Funciones en C: Estructura de una función

- ❖ En el ejemplo se puede ver una función que resuelve la sumatoria de los primeros N números enteros
- ❖ La función es de tipo entero, ya que retorna un valor de este tipo

```
int sumatoria(int n)
{
    int i, sum=0;
    for(i=0; i<=n; i++)
        sum+=i;
    return (sum);
}
```





Funciones en C: parámetros por valor

```
1  #include<stdio.h>
2  void mostrarDoble(int x);
3  int main()
4  {
5      int x;
6      scanf("%d",&x);
7      mostrarDoble(x);
8      printf("Valor de x en main(): %d",x);
9  }
10
11 void mostrarDoble(int x)
12 {
13     x=x*2;
14     printf("El doble del valor es %d \n\n",x);
15 }
```

- Al modificar la variable que recibe la función (parámetro o argumento) dentro de esta, no implica que se modifique el valor de la “variable original”, ya que a la función se le entrega un valor y no “una variable como tal” (referencia)





Funciones en C: parámetros por valor

```
1  #include<stdio.h>
2  void mostrarDoble(int x);
3  int main()
4  {
5      int x;
6      scanf("%d",&x);
7      mostrarDoble(x);
8      printf("Valor de x en main(): %d",x);
9  }
10
11 void mostrarDoble(int x)
12 {
13     x=x*2;
14     printf("El doble del valor es %d \n\n",x);
15 }
```

Variable **x** en función **main** es distinta a la variable **x** de la función **mostrarDoble**, ya que son variables locales. Al modificar **x** en **mostrarDoble**, no cambia la variable en **main** ya que la función solo recibió el valor de **x** (desde **main**)



Ejemplos 1

- a) Cree un programa en C que utilizando funciones permita validar el ingreso por teclado de tres notas
- b) Cree una función que permita validar un número **entero (int)** entre un rango de valores [A,B]
- c) Cree una función que permita validar un número **real (float)** entre un rango de valores [A,B]



Ejercicios 1

- a) Desarrolle un programa que permita ingresar 2 edades de menores de edad, 2 edades de adulto (sin pertenecer a 3ra edad) y de 2 adultos mayores. El programa debe indicar que menor de edad es mayor, que adulto es menor y que adulto mayor es mayor.





Funciones en C: arreglos

```
1 #include<stdio.h>
2 int mayor(int ar[], int n);
3 int main()
4 {
5     int v[10]={1,2,3,4,5,6,17,8,9,10};
6     printf("el mayor es: %d", mayor( v, 10 ) );
7
8 }
9 int mayor(int ar[], int n)
10 {
11     int ma=ar[0],i;
12     for(i=1;i<n;i++)
13         if(ma<ar[i])
14             ma=ar[i];
15
16     return ma;
17 }
```

Al definir como parámetro de función un vector, se puede especificar el arreglo sin dimensión (`ar[]`) y luego utilizar una variable que indique el tamaño del vector (variable `n`)

Para entregar una vector al momento de llamar la función, solo basta con indicar el nombre del arreglo (nombre de variable sin `[]`)





Funciones en C: tipo void

```
1  #include<stdio.h>
2  void imprimir_vector(int v[], int n);
3  int main()
4  {
5      int v[10]={1,2,3,4,5,6,17,8,9,10};
6
7      imprimir_vector(v,10);
8
9  }
10 void imprimir_vector(int v[], int n)
11 {
12     int i;
13     for(i=0;i<n;i++)
14         printf("[%d]",v[i]);
15 }
```

- ❖ Las funciones de tipo **void** son funciones que no “retornan” valores.
- ❖ Estas funciones son utilizadas para mostrar información o para trabajar con direcciones de memoria





Funciones en C: tipo void

```
1  #include<stdio.h>
2  void imprimir_vector(int v[], int n);
3  void llenar_vector(int v[], int n);
4  int main()
5  {
6      int v[10];
7      llenar_vector(v,10);
8      imprimir_vector(v,10);
9
10 }
11 void llenar_vector(int v[], int n)
12 {
13     int i;
14     for(i=0;i<n;i++)
15         scanf("%d",&v[i]);
16 }
17
```

- El manipular arreglos en funciones es distinto a trabajar con variables normales. En ejemplos anteriores, al entregar una variable a una función, esta última solo tomaba el valor de la variable. En el caso de los arreglos, al ser el nombre de estos una especie de “referencia” a varios espacios de memoria, si una función modifica algo de un arreglo recibido como argumento, estos cambios también se verán fuera de la función.





Funciones en C: tipo void

```
1  #include<stdio.h>
2  void imprimir_vector(int v[], int n);
3  void llenar_vector(int v[], int n);
4  int main()
5  {
6      int v[10];
7      llenar_vector(v,10);
8      imprimir_vector(v,10);
9
10 }
11 void llenar_vector(int v[], int n)
12 {
13     int i;
14     for(i=0;i<n;i++)
15         scanf("%d",&v[i]);
16 }
17
```

La función **void llenar_vector(int v[], int n)**, permite ingresar valores al arreglos desde teclado. Notar que la función en ningún momento retorna valores, aún así en la función principal **v[10]** queda con los datos ingresados.



Ejercicios 2

1. Cree una función en C que retorne el promedio simple de un vector de tipo float
2. Cree una función que retorne la cantidad de valores “X” que hay dentro de un vector numérico
3. Cree una función que muestre por pantalla la cantidad de vocales que hay en una cadena de caracteres



Ejercicios 3

1. Desarrolle un programa que permita ingresar un vector por teclado de tamaño N e indique si el vector es palíndromo, si el vector solo contiene número positivos, mostrar si la mayoría de los valores son pares. El programa ocupar al menos 3 funciones
2. El elemento mayoritario en un arreglo es el valor que aparece **más veces** que la mitad del tamaño del arreglo ($n/2$). Desarrolle una función que permita retornar el elemento mayoritario en un vector de números positivos de tamaño n , de no haberlo la función debe retornar -1.

